Aravindh Varadharaj

Backend Engineer · Go · Ingestion Pipelines · Retrieval · Agent Memory

Tiruppur, Tamil Nadu (open to relocating to Bengaluru, on-site) | aravindhvaradha@gmail.com | github.com/Aravindh97 | LinkedIn
Live resume: aravindh97.github.io/resume (ask my code questions, kill the Raft leader, flip sync.Mutex on a data race)

SUMMARY

Builder at Amazon since 2021 who moved from digital marketing into hands-on engineering by automating my team's internal workflows. I built three open-source Go systems for the problems the Tejas team works on: document ingestion with hybrid retrieval and citations, scoped long-term memory for agents, and enterprise connectors with entity resolution. Each is tested against real PostgreSQL, OpenSearch and Temporal, including crash and outage tests, and each ships with its own eval harness and baseline.

SKILLS

Languages: Go (daily: goroutines, context cancellation, net/http, generics), Python (FastAPI, Pydantic, pandas), TypeScript, SQL
Data & storage: PostgreSQL (schema design, partial unique and GIN indexes, advisory locks, logical replication, pgvector/HNSW), OpenSearch (BM25, filtered k-NN, English/Hindi analyzers)
Pipelines: Temporal (workflows, child workflows, retries, heartbeats, continue-as-new, schedules), idempotency keys, dead-letter states, CDC checkpoints, keyset incremental sync
Retrieval & agents: Hybrid search with reciprocal rank fusion, layout-aware chunking, citations, MCP servers, entity resolution
Quality: Eval harnesses (hit@k, MRR, pairwise precision/recall/F1, stale-answer rate) with baselines and held-out data, race detector, integration tests on Docker
AI coding tools: Kiro, Claude Code and Copilot every day; I use them for scaffolding and tests and review concurrency, SQL and retry logic by hand

PROJECTS

Granth: Document Ingestion & Hybrid Retrieval for Agents — GitHub

Go, Temporal, OpenSearch, PostgreSQL, Tesseract OCR, MCP

- One Temporal workflow per document (fetch, parse, OCR fallback, chunk, embed, index). Activities pass sha256 blob references and chunk ids are deterministic, so retries and replays never create duplicates.
- Corrupt, unsupported or oversized files fail once and go to a Postgres dead-letter ledger with the failing step. Transient failures retry with backoff for up to an hour; an embedding cache stops retries from paying twice.
- Stopped OpenSearch mid-ingest in a live test: the index step retried 7+ times and the document indexed after restart with no duplicate chunks.
- Hybrid search fuses BM25 and filtered k-NN with reciprocal rank fusion, falls back to BM25 when embeddings fail, and returns citations to file, page and section with a rank trace for debugging misses.
- Exposed as MCP tools with token budgets. On a 44-question labelled set: hit@5 0.977 and MRR 0.925 for hybrid vs 0.955 and 0.891 for BM25 alone.

Smriti: Long-Term Memory for LLM Agents — GitHub

Go, PostgreSQL + pgvector, Temporal, MCP

- User, team and org memories with one access rule applied in every query. Org writes need explicit permission, so one user's chat cannot change what every agent believes.
- Each fact is added, confirmed or superseded instead of appended. An advisory lock plus a partial unique index keep one current value per subject (tested with 20 concurrent writers); replaying a conversation is a no-op.
- Bitemporal rows keep history and source message, so agents can answer both "what is true now" and "what did we believe in February".
- On a 34-question multi-session scenario: 1.000 correct and 0 stale answers vs 0.676 correct and 0.176 stale for retrieval over raw chat history.

Sutra: Enterprise Connectors, Ontology & Entity Resolution — GitHub

Go, PostgreSQL logical replication (pgoutput), REST, YAML ontology, MCP

- CDC from Postgres with an exported-snapshot initial load and commit-time checkpoints. After a kill -9 mid-stream and restart, source and warehouse matched exactly (md5 over every row).
- Incremental CRM sync with keyset pagination, Retry-After handling and checkpoints written in the same transaction as each page.
- Entity resolution with GSTIN checksum validation, PAN derived from GSTIN, +91 phone normalisation and transliteration-aware names (Sri/Shree, Lakshmi/Laxmi). Two different PANs can never merge; unsure pairs go to review.
- Every golden-record value keeps lineage to its source record. On held-out labelled data, F1 0.96 to 0.996 vs 0.72 to 0.77 for GSTIN-only matching.

Also built: fairdrop (ticketing for high-demand releases); paisa (exactly-once double-entry ledger); orbit (satellite conjunction screener); relief (offline-first sync for disaster relief teams); aqcal (low-cost air sensor calibration and fault detection); foodwatch (food-stress early warning (somalia)). Each has tests, CI and measured results in its README.

Earlier projects: In github.com/Aravindh97/aravindh-impact-portfolio: OpenSchool Enterprise (Python/FastAPI learning platform with role contracts and auditable review state), FamineLens (sourced historical evidence with an uncertainty-aware scenario

model), Kindred (privacy-aware animal welfare network with a human-review queue), EcoSignal (Go RF power-plan optimiser under a coverage constraint), Nuclear Legacy & Civil Safety (Go observatory).

EXPERIENCE

Sr. Digital Marketing Associate, Amazon 2021 to Present

- Built and automated internal tools and workflows for my team, replacing repetitive manual tasks with reliable, reusable solutions.
- Worked with stakeholders to find process gaps, then designed, built and supported automation that the team adopted in daily work.
- Grew from a marketing role into hands-on development, and built a portfolio of Go and Python backend systems outside my core responsibilities.

EDUCATION

Bachelor of Engineering (B.E.) in Mechanical Engineering